EOSC 213 – Computational Methods for Geological Engineering

Homework 04 – Solution

Problem 1: Comparing Heun's Method and RK4: Amplification Factor and Stability

Consider the exponential decay model

$$x'(t) = -\lambda x(t), \qquad \lambda > 0,$$

and define

$$z := h\lambda.$$

For this problem, you will compare *Heun's method* and the *classical fourth-order Runge–Kutta method (RK4)* by deriving their amplification factors and studying their stability. Recall that Heun's method is given by

$$x_{n+1} = x_n + \frac{h}{2}(k_1 + k_2),$$

where

$$k_1 = f(t_n, x_n), \qquad k_2 = f(t_n + h,\ x_n + h\,k_1).$$

Recall also that RK4 is given by

$$x_{n+1} = x_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4),$$

where

$$\begin{aligned} k_1 &= f(t_n, x_n), \\ k_2 &= f\left(t_n + \frac{h}{2},\ x_n + \frac{h}{2}k_1\right), \\ k_3 &= f\left(t_n + \frac{h}{2},\ x_n + \frac{h}{2}k_2\right), \\ k_4 &= f(t_n + h,\ x_n + h\,k_3). \end{aligned}$$

(a) Derive the amplification factor for Heun's method

Apply Heun's method to

$$x'(t) = -\lambda x(t)$$

and show that the update can be written as

$$x_{n+1} = g_H(z)\,x_n,$$

where

$$g_H(z) = 1 - z + \frac{z^2}{2}.$$

(b) Derive the amplification factor for RK4

Apply RK4 to

$$x'(t) = -\lambda x(t)$$

and show that the update can be written as

$$x_{n+1} = g_{RK4}(z)\, x_n,$$

where

$$g_{RK4}(z) = 1 - z + \frac{z^2}{2} - \frac{z^3}{6} + \frac{z^4}{24}.$$

(c) Plot and interpret

For stability, the amplification factor $|g(z)| < 1$. Plot $|g_H(z)|$ and $|g_{RK4}(z)|$ for $0 \le z \le 4$ on the same graph, and use the plots to compare the two methods from the point of view of stability. Briefly explain which method remains stable for a larger interval of z step sizes, and which method better approximates the exponential decay factor. Your final solution should consist of a plot showing the stability regions of both methods along with the explanations. The plot can be either drawn via a graphing tool like Desmos (https://www.desmos.com/calculator) or Matplotlib in Python.

Solution 1

We consider the exponential decay model

$$x'(t) = -\lambda x(t), \qquad \lambda > 0,$$

and define

$$z := h\lambda.$$

(a) Derive the amplification factor for Heun's method

For this ODE,

$$f(t, x) = -\lambda x.$$

Hence

$$k_1 = f(t_n, x_n) = -\lambda x_n.$$

Next,

$$x_n + h\, k_1 = x_n + h(-\lambda x_n) = x_n - h\lambda x_n = (1 - z)x_n.$$

Therefore,

$$k_2 = f(t_n + h,\ x_n + h\,k_1) = -\lambda(1-z)x_n.$$

Now Heun's method gives

$$x_{n+1} = x_n + \frac{h}{2}(k_1 + k_2).$$

Substituting the expressions for k_1 and k_2,

$$x_{n+1} = x_n + \frac{h}{2}\left[-\lambda x_n - \lambda(1-z)x_n\right].$$

Factor out $-\lambda x_n$:

$$x_{n+1} = x_n - \frac{h\lambda}{2}(2-z)x_n.$$

Since $z = h\lambda$,

$$x_{n+1} = x_n - \frac{z}{2}(2-z)x_n = \left(1 - z + \frac{z^2}{2}\right)x_n.$$

Thus the amplification factor for Heun's method is

$$g_H(z) = 1 - z + \frac{z^2}{2}.$$

(b) Derive the amplification factor for RK4

Again, for this ODE,

$$f(t,x) = -\lambda x.$$

The RK4 stages are:

$$k_1 = -\lambda x_n.$$

Next,

$$x_n + \frac{h}{2}k_1 = x_n + \frac{h}{2}(-\lambda x_n) = \left(1 - \frac{z}{2}\right)x_n,$$

so

$$k_2 = -\lambda\left(1 - \frac{z}{2}\right)x_n.$$

Now

$$x_n + \frac{h}{2}k_2 = x_n + \frac{h}{2}\left[-\lambda\left(1 - \frac{z}{2}\right)x_n\right] = \left(1 - \frac{z}{2} + \frac{z^2}{4}\right)x_n,$$

hence

$$k_3 = -\lambda\left(1 - \frac{z}{2} + \frac{z^2}{4}\right)x_n.$$

Next,

$$x_n + h\,k_3 = x_n + h\left[-\lambda\left(1 - \frac{z}{2} + \frac{z^2}{4}\right)x_n\right] = \left(1 - z + \frac{z^2}{2} - \frac{z^3}{4}\right)x_n,$$

so

$$k_4 = -\lambda \left(1 - z + \frac{z^2}{2} - \frac{z^3}{4}\right) x_n.$$

Now substitute into RK4:

$$x_{n+1} = x_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4).$$

First compute the weighted sum:

$$k_1 + 2k_2 + 2k_3 + k_4 = -\lambda x_n - 2\lambda \left(1 - \frac{z}{2}\right) x_n - 2\lambda \left(1 - \frac{z}{2} + \frac{z^2}{4}\right) x_n - \lambda \left(1 - z + \frac{z^2}{2} - \frac{z^3}{4}\right) x_n.$$

Collecting terms gives

$$k_1 + 2k_2 + 2k_3 + k_4 = -\lambda \left(6 - 3z + z^2 - \frac{z^3}{4}\right) x_n.$$

Therefore,

$$x_{n+1} = x_n + \frac{h}{6}\left[-\lambda \left(6 - 3z + z^2 - \frac{z^3}{4}\right) x_n\right].$$

Using $h\lambda = z$,

$$x_{n+1} = x_n - \frac{z}{6}\left(6 - 3z + z^2 - \frac{z^3}{4}\right) x_n.$$

Expanding,

$$x_{n+1} = \left(1 - z + \frac{z^2}{2} - \frac{z^3}{6} + \frac{z^4}{24}\right) x_n.$$

Thus the amplification factor for RK4 is

$$g_{RK4}(z) = 1 - z + \frac{z^2}{2} - \frac{z^3}{6} + \frac{z^4}{24}.$$

(c) Plot and interpret

From the earlier parts, the amplification factors are

$$g_H(z) = 1 - z + \frac{z^2}{2},$$

and

$$g_{RK4}(z) = 1 - z + \frac{z^2}{2} - \frac{z^3}{6} + \frac{z^4}{24}.$$

For stability on the test equation

$$x'(t) = -\lambda x,$$

we require

$$|g(z)| < 1.$$

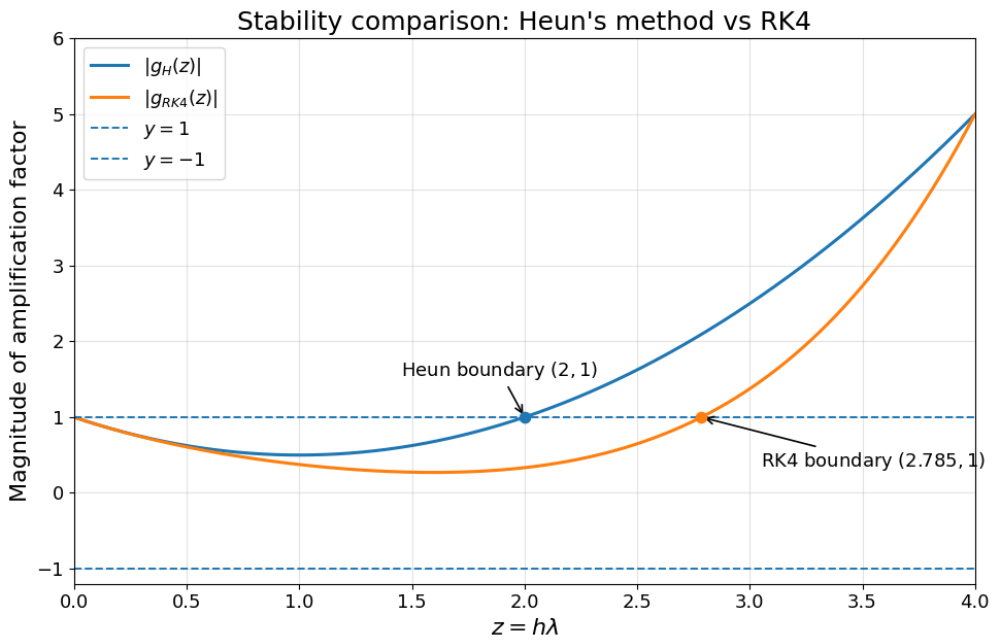

Figure 1: Stability comparison between Heun's and RK4 methods.

To compare the two methods, we plot

$$|g_H(z)| \qquad \text{and} \qquad |g_{RK4}(z)|$$

for $0 \leq z \leq 4$, together with the horizontal line

$$|g(z)| = 1.$$

From the plot, we observe the following: For Heun's method, the curve $|g_H(z)|$ remains below 1 for

$$0 \leq z < 2.$$

At

$$z = 2,$$

we have

$$|g_H(2)| = 1,$$

so $z = 2$ is the stability boundary for Heun's method. For RK4, the curve $|g_{RK4}(z)|$ remains below 1 for a larger interval,

$$0 \leq z < 2.785 \quad \text{(approximately)}.$$

At about

$$z \approx 2.785,$$

we have

$$|g_{RK4}(z)| = 1,$$

so this is the stability boundary for RK4. Therefore, RK4 remains stable for a larger interval of z-values than Heun's method. Also, RK4 better approximates the exact exponential decay factor

$$e^{-z},$$

because its amplification factor

$$g_{RK4}(z) = 1 - z + \frac{z^2}{2} - \frac{z^3}{6} + \frac{z^4}{24}$$

matches more terms of the Taylor expansion of e^{-z} than Heun's method does. Heun's method matches the expansion only up to the z^2 term, while RK4 matches it up to the z^4 term. Hence:

- Heun's method is stable for $0 \leq z < 2$,
- RK4 is stable for approximately $0 \leq z < 2.785$,
- RK4 both remains stable for larger step sizes and gives a more accurate approximation to the exponential decay factor.

Problem 2: Analytical Parameter Estimation for a Cooling Model

Consider Newton's law of cooling

$$T'(t) = -k\big(T(t) - T_{\text{env}}\big), \qquad k > 0,$$

where T_{env} is the known ambient temperature and k is an unknown parameter. Suppose the initial temperature $T(0) = T_0$ is known. Assume that we observe the temperature at times

$$t_1, t_2, \ldots, t_N,$$

and obtain measurements

$$T_1^{\text{obs}}, T_2^{\text{obs}}, \ldots, T_N^{\text{obs}}.$$

In this problem, you will derive an analytical estimator for the unknown parameter k.

(a) Solve the ODE exactly

Solve the initial value problem and show that the exact solution can be written as

$$T(t) = T_{\text{env}} + \big(T_0 - T_{\text{env}}\big)e^{-kt}.$$

(b) Transform the model into a linear relation

Assume that $T_i^{\text{obs}} > T_{\text{env}}$ for all i, so that the logarithm below is defined. Show that the model implies

$$\ln\left(\frac{T(t) - T_{\text{env}}}{T_0 - T_{\text{env}}}\right) = -kt.$$

Hence define

$$y_i := \ln\left(\frac{T_i^{\text{obs}} - T_{\text{env}}}{T_0 - T_{\text{env}}}\right),$$

and show that the parameter estimation problem becomes fitting the straight line

$$y_i \approx -k\, t_i.$$

(c) Build the least-squares loss

Using the transformed variables from part (b), define the least-squares loss

$$L(k) = \frac{1}{N}\sum_{i=1}^{N}\big(y_i + kt_i\big)^2.$$

Show that $L(k)$ is a quadratic function of k.

(d) Find the best-fit parameter analytically

Differentiate $L(k)$ with respect to k, set the derivative equal to zero, and derive a closed-form formula for the least-squares estimator $\hat{k}$ which minimizes the loss $L(k)$.

(e) Interpret the result

Briefly explain why this problem becomes easy to solve analytically after the logarithmic transformation, and why such transformations are useful in parameter estimation.

Solution 2

We consider the ODE

$$T'(t) = -k\big(T(t) - T_{\text{env}}\big), \qquad k > 0,$$

with initial condition

$$T(0) = T_0,$$

where T_{env} and T_0 are known, and k is unknown.

(a) Solve the ODE exactly

We first rewrite the equation as

$$T'(t) + kT(t) = kT_{\text{env}}.$$

A simpler way is to define

$$u(t) := T(t) - T_{\text{env}}.$$

Since T_{env} is constant, we have

$$u'(t) = T'(t).$$

Substituting into the ODE gives

$$u'(t) = -ku(t).$$

This is a separable equation whose solution is

$$u(t) = Ce^{-kt}.$$

Since

$$u(t) = T(t) - T_{\text{env}},$$

we get

$$T(t) - T_{\text{env}} = Ce^{-kt}.$$

Now use the initial condition $T(0) = T_0$:

$$T_0 - T_{\text{env}} = C.$$

Therefore,

$$T(t) = T_{\text{env}} + \big(T_0 - T_{\text{env}}\big)e^{-kt}.$$

(b) Transform the model into a linear relation

From part (a), we have

$$T(t) - T_{\text{env}} = \big(T_0 - T_{\text{env}}\big)e^{-kt}.$$

Divide both sides by $T_0 - T_{\text{env}}$:

$$\frac{T(t) - T_{\text{env}}}{T_0 - T_{\text{env}}} = e^{-kt}.$$

Now take the natural logarithm:

$$\ln\left(\frac{T(t) - T_{\text{env}}}{T_0 - T_{\text{env}}}\right) = -kt.$$

For each observation time t_i, define

$$y_i := \ln\left(\frac{T_i^{\text{obs}} - T_{\text{env}}}{T_0 - T_{\text{env}}}\right).$$

Then the model predicts

$$y_i \approx -k\, t_i.$$

So after the logarithmic transformation, the parameter estimation problem becomes a straight-line

fitting problem through the origin, with slope $-k$.

(c) Build the least-squares loss

The transformed model is

$$y_i \approx -kt_i.$$

So the residual at time t_i is

$$r_i(k) = y_i - (-kt_i) = y_i + kt_i.$$

Therefore the least-squares loss is

$$L(k) = \frac{1}{N}\sum_{i=1}^{N}\big(y_i + kt_i\big)^2.$$

Expand the square:

$$L(k) = \frac{1}{N}\sum_{i=1}^{N}\left(y_i^2 + 2kt_iy_i + k^2t_i^2\right).$$

Distribute the summation:

$$L(k) = \left(\frac{1}{N}\sum_{i=1}^{N} y_i^2\right) + 2k\left(\frac{1}{N}\sum_{i=1}^{N} t_iy_i\right) + k^2\left(\frac{1}{N}\sum_{i=1}^{N} t_i^2\right).$$

The terms $\frac{1}{N}\sum_{i=1}^{N} y_i^2$, $\frac{1}{N}\sum_{i=1}^{N} t_iy_i$ and $\frac{1}{N}\sum_{i=1}^{N} t_i^2$ are constants and completely determined from the observed data. Thus $L(k)$ is a quadratic function of k.

(d) Find the best-fit parameter analytically

Differentiate $L(k)$ with respect to k:

$$\frac{dL}{dk} = 2\left(\frac{1}{N}\sum_{i=1}^{N} t_iy_i\right) + 2k\left(\frac{1}{N}\sum_{i=1}^{N} t_i^2\right).$$

Set this equal to zero:

$$2\left(\frac{1}{N}\sum_{i=1}^{N} t_iy_i\right) + 2k\left(\frac{1}{N}\sum_{i=1}^{N} t_i^2\right) = 0.$$

Divide by 2 and multiply by N:

$$\sum_{i=1}^{N} t_iy_i + k\sum_{i=1}^{N} t_i^2 = 0.$$

Solve for k:

$$\hat{k} = -\frac{\sum_{i=1}^{N} t_iy_i}{\sum_{i=1}^{N} t_i^2}.$$

Now substitute the definition of y_i:

$$\hat{k} = -\frac{\sum_{i=1}^{N} t_i \ln\left(\dfrac{T_i^{\text{obs}} - T_{\text{env}}}{T_0 - T_{\text{env}}}\right)}{\sum_{i=1}^{N} t_i^2}.$$

This is the closed-form least-squares estimator for the cooling-rate parameter.

(e) Interpret the result

The original model

$$T(t) = T_{\text{env}} + \big(T_0 - T_{\text{env}}\big)e^{-kt}$$

depends on k in a nonlinear way because k appears inside the exponential. After taking logarithms, the model becomes

$$y_i \approx -kt_i,$$

which is linear in the unknown parameter k. After writing the problem in this form, the least-squares loss becomes a simple quadratic function, and the minimizer can be found exactly by differentiation. This is useful because some parameter estimation problems that look nonlinear at first can be transformed into much simpler linear fitting problems. Such transformations make the analysis easier and can lead to closed-form formulas for the best-fit parameter.

Problem 3: Analytical Parameter Estimation for a Two-Parameter ODE Model

Consider the initial value problem

$$y'(t) = -y(t) + b, \qquad y(0) = a,$$

where a and b are unknown parameters.

Suppose we observe the solution at times

$$t_1, t_2, \ldots, t_N,$$

and obtain measurements

$$y_1^{\text{obs}}, y_2^{\text{obs}}, \ldots, y_N^{\text{obs}}.$$

In this problem, you will derive the exact solution, formulate a least-squares parameter estimation problem for a and b, solve it analytically, and visualize the result.

(a) Solve the ODE exactly

Show that the solution of

$$y'(t) = -y(t) + b, \qquad y(0) = a,$$

is

$$y(t) = ae^{-t} + b\big(1 - e^{-t}\big).$$

(b) Write the model in linear form

Show that for each observation time t_i, the model prediction can be written as

$$y(t_i) = a\phi_i + b\psi_i,$$

where

$$\phi_i = e^{-t_i}, \qquad \psi_i = 1 - e^{-t_i}.$$

(c) Build the least-squares loss

Define the least-squares loss (here, we have defined the least-squares loss as a sum instead of mean; this is fine since the sum only differs from the mean by a fixed multiplicative constant),

$$L(a, b) = \sum_{i=1}^{N} \Big(a\phi_i + b\psi_i - y_i^{\text{obs}}\Big)^2.$$

Expand this expression and show that it is a quadratic function of a and b.

(d) Derive the normal equations

Differentiate $L(a, b)$ with respect to a and b, set both partial derivatives equal to zero, and show that the best-fit parameters satisfy

$$a\sum_{i=1}^{N} \phi_i^2 + b\sum_{i=1}^{N} \phi_i\psi_i = \sum_{i=1}^{N} \phi_i y_i^{\text{obs}},$$

$$a\sum_{i=1}^{N} \phi_i\psi_i + b\sum_{i=1}^{N} \psi_i^2 = \sum_{i=1}^{N} \psi_i y_i^{\text{obs}}.$$

(e) Solve for the best-fit parameters

Solve the 2×2 linear system from part (d) to obtain explicit formulas for the least-squares estimators $\hat{a}$ and $\hat{b}$.

(f) Make plots and interpret

Use the following code to import the data $\{(t_i, y_i^{\text{obs}})\}_{i=1}^{N}$ from the file `data_problem03.pt` to Python variables, `t_obs` and `y_obs`.

```
import torch
data = torch.load('data_problem03.pt')
t_obs = data['t_obs']
```

```
y_obs = data['y_obs']
```

(i) a scatter plot of the observed data together with the best-fit curve

$$y(t) = \hat{a}e^{-t} + \hat{b}(1 - e^{-t}),$$

(ii) a contour plot of the loss function $L(a, b)$ in the (a, b)-plane.

Briefly explain how these plots help us understand the parameter estimation problem. You only need to submit the plots and the brief explanation. There is no need to include your code for this problem.

Solution 3

We consider the ODE

$$y'(t) = -y(t) + b, \qquad y(0) = a,$$

where a and b are unknown parameters.

(a) Solve the ODE exactly

We want to solve

$$y'(t) = -y(t) + b, \qquad y(0) = a.$$

Define a new variable

$$u(t) = y(t) - b.$$

Since b is a constant, we have

$$u'(t) = y'(t).$$

Also,

$$y(t) = u(t) + b.$$

Substituting into the ODE gives

$$u'(t) = -(u(t) + b) + b = -u(t).$$

So the transformed equation is

$$u'(t) = -u(t).$$

Its general solution is

$$u(t) = Ce^{-t},$$

where C is a constant. Since $u(t) = y(t) - b$, it follows that

$$y(t) - b = Ce^{-t},$$

and hence

$$y(t) = b + Ce^{-t}.$$

Now use the initial condition $y(0) = a$:

$$a = b + C.$$

So

$$C = a - b.$$

Substituting back, we obtain

$$y(t) = b + (a - b)e^{-t}.$$

Rearranging,

$$y(t) = ae^{-t} + b(1 - e^{-t}).$$

Hence the exact solution is

$$\boxed{y(t) = ae^{-t} + b(1 - e^{-t}).}$$

(b) Write the model in linear form

At time t_i, the model prediction is

$$y(t_i) = ae^{-t_i} + b(1 - e^{-t_i}).$$

Define

$$\phi_i = e^{-t_i}, \qquad \psi_i = 1 - e^{-t_i}.$$

Then

$$y(t_i) = a\phi_i + b\psi_i.$$

So the model is linear in the unknown parameters a and b.

(c) Build the least-squares loss

The residual at time t_i is

$$r_i(a, b) = a\phi_i + b\psi_i - y_i^{\text{obs}}.$$

Therefore the least-squares loss is

$$L(a, b) = \sum_{i=1}^{N} \left(a\phi_i + b\psi_i - y_i^{\text{obs}} \right)^2.$$

Expanding the square,

$$L(a, b) = \sum_{i=1}^{N} \left(a^2\phi_i^2 + 2ab\,\phi_i\psi_i + b^2\psi_i^2 - 2a\phi_i y_i^{\text{obs}} - 2b\psi_i y_i^{\text{obs}} + (y_i^{\text{obs}})^2 \right).$$

So

$$L(a,b) = a^2 \sum_{i=1}^{N} \phi_i^2 + 2ab \sum_{i=1}^{N} \phi_i \psi_i + b^2 \sum_{i=1}^{N} \psi_i^2 - 2a \sum_{i=1}^{N} \phi_i y_i^{\text{obs}} - 2b \sum_{i=1}^{N} \psi_i y_i^{\text{obs}} + \sum_{i=1}^{N} (y_i^{\text{obs}})^2.$$

Thus $L(a,b)$ is a quadratic function of a and b.

(d) Derive the normal equations

Differentiate $L(a,b)$ with respect to a:

$$\frac{\partial L}{\partial a} = 2a \sum_{i=1}^{N} \phi_i^2 + 2b \sum_{i=1}^{N} \phi_i \psi_i - 2 \sum_{i=1}^{N} \phi_i y_i^{\text{obs}}.$$

Set this equal to zero:

$$a \sum_{i=1}^{N} \phi_i^2 + b \sum_{i=1}^{N} \phi_i \psi_i = \sum_{i=1}^{N} \phi_i y_i^{\text{obs}}.$$

Now differentiate with respect to b:

$$\frac{\partial L}{\partial b} = 2a \sum_{i=1}^{N} \phi_i \psi_i + 2b \sum_{i=1}^{N} \psi_i^2 - 2 \sum_{i=1}^{N} \psi_i y_i^{\text{obs}}.$$

Set this equal to zero:

$$a \sum_{i=1}^{N} \phi_i \psi_i + b \sum_{i=1}^{N} \psi_i^2 = \sum_{i=1}^{N} \psi_i y_i^{\text{obs}}.$$

These are the normal equations.

(e) Solve for the best-fit parameters

From part (d), the normal equations are

$$a \sum_{i=1}^{N} \phi_i^2 + b \sum_{i=1}^{N} \phi_i \psi_i = \sum_{i=1}^{N} \phi_i y_i^{\text{obs}},$$

$$a \sum_{i=1}^{N} \phi_i \psi_i + b \sum_{i=1}^{N} \psi_i^2 = \sum_{i=1}^{N} \psi_i y_i^{\text{obs}}.$$

For simplicity, define

$$S_{\phi\phi} = \sum_{i=1}^{N} \phi_i^2, \qquad S_{\phi\psi} = \sum_{i=1}^{N} \phi_i \psi_i, \qquad S_{\psi\psi} = \sum_{i=1}^{N} \psi_i^2,$$

$$S_{\phi y} = \sum_{i=1}^{N} \phi_i y_i^{\text{obs}}, \qquad S_{\psi y} = \sum_{i=1}^{N} \psi_i y_i^{\text{obs}}.$$

Then the system becomes

$$S_{\phi\phi}a + S_{\phi\psi}b = S_{\phi y},$$

$$S_{\phi\psi}a + S_{\psi\psi}b = S_{\psi y}.$$

We now solve this system by elimination. First, multiply the first equation by $S_{\psi\psi}$:

$$S_{\psi\psi}S_{\phi\phi}a + S_{\psi\psi}S_{\phi\psi}b = S_{\psi\psi}S_{\phi y}.$$

Next, multiply the second equation by $S_{\phi\psi}$:

$$S_{\phi\psi}S_{\phi\psi}a + S_{\phi\psi}S_{\psi\psi}b = S_{\phi\psi}S_{\psi y}.$$

Now subtract the second of these equations from the first. The b-terms cancel, giving

$$\left(S_{\phi\phi}S_{\psi\psi} - S_{\phi\psi}^2\right)a = S_{\psi\psi}S_{\phi y} - S_{\phi\psi}S_{\psi y}.$$

Hence

$$\hat{a} = \frac{S_{\psi\psi}S_{\phi y} - S_{\phi\psi}S_{\psi y}}{S_{\phi\phi}S_{\psi\psi} - S_{\phi\psi}^2}.$$

Now solve for b. Multiply the first equation by $S_{\phi\psi}$:

$$S_{\phi\psi}S_{\phi\phi}a + S_{\phi\psi}^2 b = S_{\phi\psi}S_{\phi y}.$$

Multiply the second equation by $S_{\phi\phi}$:

$$S_{\phi\phi}S_{\phi\psi}a + S_{\phi\phi}S_{\psi\psi}b = S_{\phi\phi}S_{\psi y}.$$

Subtract the first of these equations from the second. The a-terms cancel, giving

$$\left(S_{\phi\phi}S_{\psi\psi} - S_{\phi\psi}^2\right)b = S_{\phi\phi}S_{\psi y} - S_{\phi\psi}S_{\phi y}.$$

Therefore,

$$\hat{b} = \frac{S_{\phi\phi}S_{\psi y} - S_{\phi\psi}S_{\phi y}}{S_{\phi\phi}S_{\psi\psi} - S_{\phi\psi}^2}.$$

Substituting back the definitions of the sums, we obtain

$$\hat{a} = \frac{\left(\sum \psi_i^2\right)\left(\sum \phi_i y_i^{\text{obs}}\right) - \left(\sum \phi_i\psi_i\right)\left(\sum \psi_i y_i^{\text{obs}}\right)}{\left(\sum \phi_i^2\right)\left(\sum \psi_i^2\right) - \left(\sum \phi_i\psi_i\right)^2},$$

$$\hat{b} = \frac{\left(\sum \phi_i^2\right)\left(\sum \psi_i y_i^{\text{obs}}\right) - \left(\sum \phi_i\psi_i\right)\left(\sum \phi_i y_i^{\text{obs}}\right)}{\left(\sum \phi_i^2\right)\left(\sum \psi_i^2\right) - \left(\sum \phi_i\psi_i\right)^2}.$$

These are the least-squares estimates of the parameters a and b.

(f) Make plots and interpret

The fitted curve is

$$y(t) = \hat{a}e^{-t} + \hat{b}(1 - e^{-t}).$$

A scatter plot of the observed data together with this fitted curve shows how well the model matches the measurements across time.

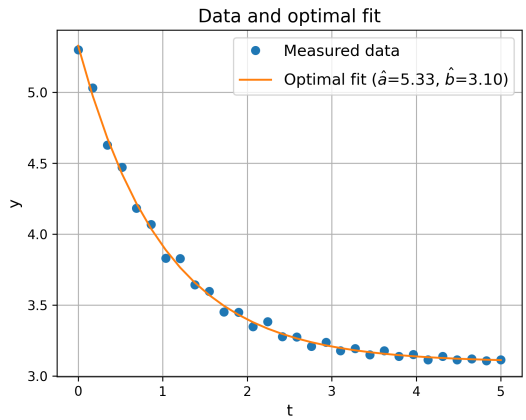

Figure 2: Fit to the data with the optimal parameters $\hat{a}$ and $\hat{b}$.

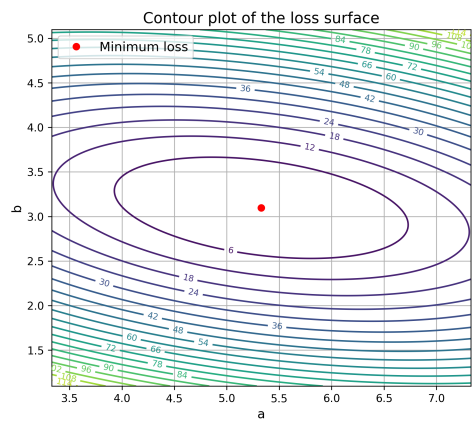

Figure 3: Loss surface $L(a, b)$ as a function of the parameters a and b.

A contour plot of the loss function $L(a, b)$ in the (a, b)-plane shows the geometry of the optimization problem. Since $L(a, b)$ is a quadratic function, its contours are ellipses, and the minimizer $(\hat{a}, \hat{b})$ appears at the center of the nested contour curves. These plots are useful because:

- the data-and-fit plot shows the quality of the model fit in physical space,
- the contour plot shows how the loss changes as the parameters vary,
- together they connect the idea of parameter estimation with the geometry of optimization.

The code to make these plots is given below for your reference (not graded):

```
import torch
import matplotlib.pyplot as plt

def get_phi_psi(t):
    phi = torch.exp(-t)
    psi = 1 - phi
    return phi, psi

def y(t, a, b):
    phi, psi = get_phi_psi(t)
    return a * phi + b * psi


```

```python
def get_optimal_params(t, y_obs):
    phi, psi = get_phi_psi(t)

    common_denominator = torch.sum(phi**2) * torch.sum(psi**2) - torch.sum(psi * phi)**2

    a_hat = (torch.sum(psi**2) * torch.sum(phi * y_obs) - torch.sum(phi*psi) * torch.sum(psi * y_obs))/common_denominator
    b_hat = (torch.sum(phi**2) * torch.sum(psi * y_obs) - torch.sum(phi*psi) * torch.sum(phi * y_obs))/common_denominator

    return a_hat, b_hat

def loss_fn(a, b, t, y_obs):
    y_pred = y(t, a, b)
    return torch.sum((y_pred - y_obs)**2)

# %%
# ---------------------------------------------------
# Import data and compute optimal parameters
# Plot the data and the optimal fit
# ---------------------------------------------------

data = torch.load('data_problem03.pt')
t_obs = data['t_obs']
y_obs = data['y_obs']
y_obs, t_obs = data['y_obs'], data['t_obs']
a_hat, b_hat = get_optimal_params(t_obs, y_obs)
y_pred = y(t_obs, a_hat, b_hat)

plt.plot(t_obs, y_obs, 'o', label='Measured data')
plt.plot(t_obs, y_pred, label=fr'Optimal fit ($\hat{{a}}$={a_hat:.2f}, $\hat{{b}}$={b_hat:.2f})')
plt.legend(fontsize=12)
plt.xlabel('t', fontsize=12)
plt.ylabel('y', fontsize=12)
plt.title('Data and optimal fit', fontsize=14)
plt.grid(True)
plt.savefig('data_and_fit_problem03.png', dpi=300)
plt.show()

# ---------------------------------------------------
# Contour plot of the loss surface
# ---------------------------------------------------

a_vals = torch.linspace(a_hat - 2.0, a_hat + 2.0, 200)
b_vals = torch.linspace(b_hat - 2.0, b_hat + 2.0, 200)
```

```python

A, B = torch.meshgrid(a_vals, b_vals, indexing='ij')
L = torch.zeros_like(A)

for i in range(A.shape[0]):
    for j in range(A.shape[1]):
        L[i, j] = loss_fn(A[i, j], B[i, j], t_obs, y_obs)

plt.figure(figsize=(7, 6))
contours = plt.contour(A.numpy(), B.numpy(), L.numpy(), levels=20)
plt.clabel(contours, inline=True, fontsize=8)
plt.plot(a_hat.item(), b_hat.item(), 'ro', label='Minimum loss')
plt.xlabel('a', fontsize=12)
plt.ylabel('b', fontsize=12)
plt.title('Contour plot of the loss surface', fontsize=14)
plt.legend(fontsize=12)
plt.grid(True)
plt.savefig('loss_surface_problem03.png', dpi=300)
plt.show()
```

Problem 4: Gradient Descent on a Quadratic Loss Function

Consider the quadratic loss function

$$L(\theta) = \frac{1}{2}a\theta^2 - b\theta + c,$$

where $a > 0$, and b, c are constants and θ is the learnable parameter. In this problem, you will analyze gradient descent applied to this loss function. Recall that the gradient descent iteration is

$$\theta_{n+1} = \theta_n - \eta L'(\theta_n),$$

where $\eta > 0$ is the learning rate.

(a) Find the minimizer of the loss

Differentiate $L(\theta)$, and show that the unique minimizer is

$$\theta^* = \frac{b}{a}.$$

(b) Derive the gradient descent update

Show that gradient descent applied to $L(\theta)$ gives

$$\theta_{n+1} = (1 - a\eta)\theta_n + \eta b.$$

(c) Derive the error iteration

Define the error

$$e_n := \theta_n - \theta^*.$$

Show that the errors satisfy

$$e_{n+1} = (1 - a\eta)e_n.$$

(d) Determine the convergence condition

Use part (c) to show that gradient descent converges to θ^* if and only if

$$|1 - a\eta| < 1.$$

Hence show that convergence occurs when

$$0 < \eta < \frac{2}{a}.$$

(e) Interpret different choices of learning rate

Briefly explain what happens in each of the following cases:

$$0 < \eta < \frac{1}{a}, \qquad \eta = \frac{1}{a}, \qquad \frac{1}{a} < \eta < \frac{2}{a}, \qquad \eta > \frac{2}{a}.$$

(f) Connect with optimization practice

Explain why this problem helps us understand the importance of choosing a good learning rate in gradient descent.

Solution 4

We consider the quadratic loss

$$L(\theta) = \frac{1}{2}a\theta^2 - b\theta + c, \qquad a > 0.$$

(a) Find the minimizer of the loss

Differentiate $L(\theta)$ with respect to θ:

$$L'(\theta) = a\theta - b.$$

To find a critical point, set

$$L'(\theta) = 0.$$

This gives

$$a\theta - b = 0,$$

so

$$\theta^* = \frac{b}{a}.$$

Since

$$L''(\theta) = a > 0,$$

the function is strictly convex, so this critical point is the unique minimizer (and not the maximizer).

(b) Derive the gradient descent update

The gradient descent iteration is

$$\theta_{n+1} = \theta_n - \eta L'(\theta_n).$$

Using

$$L'(\theta_n) = a\theta_n - b,$$

we get

$$\theta_{n+1} = \theta_n - \eta(a\theta_n - b).$$

Distribute:

$$\theta_{n+1} = \theta_n - a\eta\theta_n + \eta b.$$

Hence

$$\theta_{n+1} = (1 - a\eta)\theta_n + \eta b.$$

(c) Derive the error iteration

Define

$$e_n = \theta_n - \theta^*.$$

Then

$$e_{n+1} = \theta_{n+1} - \theta^*.$$

Using the update from part (b),

$$e_{n+1} = (1 - a\eta)\theta_n + \eta b - \theta^*.$$

Now substitute

$$\theta^* = \frac{b}{a}.$$

Then

$$\eta b = \eta a\theta^*,$$

so

$$e_{n+1} = (1 - a\eta)\theta_n + a\eta\theta^* - \theta^*.$$

Factor the θ^* terms:

$$e_{n+1} = (1 - a\eta)\theta_n - (1 - a\eta)\theta^*.$$

Thus

$$e_{n+1} = (1 - a\eta)(\theta_n - \theta^*).$$

Therefore

$$e_{n+1} = (1 - a\eta)e_n.$$

(d) Determine the convergence condition

From part (c),

$$e_n = (1 - a\eta)^n e_0.$$

For gradient descent to converge to the minimizer, we need

$$e_n \to 0 \qquad \text{as } n \to \infty.$$

Practically, this means that the error e_n should go to zero given sufficiently large number of parameter updates. This happens if and only if

$$|1 - a\eta| < 1.$$

Now solve this inequality:

$$-1 < 1 - a\eta < 1.$$

Subtract 1 throughout:

$$-2 < -a\eta < 0.$$

Multiply by -1, reversing the inequalities:

$$0 < a\eta < 2.$$

Since $a > 0$, divide by a:

$$0 < \eta < \frac{2}{a}.$$

So gradient descent converges if and only if

$$0 < \eta < \frac{2}{a}.$$

(e) Interpret different choices of learning rate

We examine the factor

$$1 - a\eta.$$

Case 1: $0 < \eta < \frac{1}{a}$. Then

$$0 < 1 - a\eta < 1.$$

So the errors satisfy

$$e_{n+1} = (1 - a\eta)e_n$$

with a positive factor smaller than 1. The iterates approach the minimizer smoothly from one side without oscillation.

Case 2: $\eta = \frac{1}{a}$**.** Then

$$1 - a\eta = 0.$$

So

$$e_{n+1} = 0.$$

This means gradient descent reaches the exact minimizer in one step.

Case 3: $\frac{1}{a} < \eta < \frac{2}{a}$**.** Then

$$-1 < 1 - a\eta < 0.$$

So the errors alternate sign while decreasing in magnitude. This means the iterates oscillate around the minimizer, but still converge.

Case 4: $\eta > \frac{2}{a}$**.** Then

$$|1 - a\eta| > 1.$$

So the errors grow in magnitude instead of shrinking. Gradient descent becomes unstable and diverges.

(f) Connect with optimization practice

This problem shows that even for a very simple quadratic loss, the learning rate η has a major effect on the behavior of gradient descent. If η is too small, convergence is slow. If η is chosen well, convergence is fast. If η is too large, the method becomes unstable and may diverge.

This helps explain why choosing an appropriate learning rate is one of the most important practical decisions in optimization.